

CM2005 Object Oriented Programming

Course Description

There are several programming paradigms in computer science. One of the most important, especially when creating large software projects, is object-oriented programming, or OOP. OOP allows the integration of data and processes on that data into discrete software modules. Learning about OOP will allow you to develop more modularised, more complex software designs. It will also let you understand better how many existing software libraries and systems have been designed. In the later stages of the programme, OOP is used extensively.

This module aims to provide you with an object-oriented programming skill set. You will learn what objects and classes are and how to write classes. You will see how objects can interact with each other, including defining and implementing interfaces to control the interaction. You will learn how to use inheritance to inherit and extend functionality from parent classes. You will learn how to write code according to style guidelines and how to write formal code documentation.

1. Text I/O and functions
2. Using classes and variables to model data
3. File I/O, exception handling and algorithms
4. Writing and testing an algorithm
5. Object interactions
6. Libraries, toolkits, frameworks and widgets
7. Event driven programming and inheritance
8. Refactoring and class design
9. Initialiser lists, constructors and threads
10. Advanced class and user interfaces

Course Goals and Objectives

Upon successful completion of this course, you will be able to:

1. Understand and explain the key principles of object oriented programming
2. Choose appropriate basic data types to represent different data
3. Write classes with data and functions
4. Explain the purpose of interfaces and write classes that implement specific interfaces
5. Use inheritance to implement a hierarchy of classes
6. Write formal code documentation and write code following style guidelines

Textbook and Readings

Specific essential readings for this course will be taken from the following textbook:

- **Beginning C++17: From Novice to Professional** Horton I., Van Weert P. (2018): Apress

The specific pages for the reading activities will be given on the platform, and there is no need to read beyond the recommended pages.

In addition to the textbook, there are additional reading activities written by the course author, some of which involve coding exercises.

There will also be discussion prompts asking you to do some independent research using online sources.

Course Outline

The course consists of ten topics divided into 20 weeks that focus on key concepts.

Topic 1: Text I/O and functions: the main menu	Key concepts: You will learn about the C++ edit, compile and run cycle, and text I/O. Learning outcomes: • C++ edit, compile and run cycle • Text I/O • Functions
Topic 2: Using classes and variables to model data: the OrderBookEntry class	Key concepts: You will learn about the basic data types (numbers and strings) and classes and data. Learning outcomes: • Select appropriate data types to represent a dataset in a C++ program • Describe how a class can be used to combine multiple pieces of data into one unit • Write a class with functions

Topic 3: File I/O, exception handling and algorithms: the CSVReader class	Key concepts: You will learn about translating pseudocode to C++, exception handling and file I/O. Learning outcomes: • Convert pseudocode algorithms involving iteration, logic and string processing into working C++ code • Use exception handling to gracefully recover when processing unreliable data • Read text data from a file using the getline function
Topic 4: Writing and testing an algorithm: taking orders and the order matching engine	Key concepts: You will learn about iterating over vectors and exception handling. Learning outcomes: • Write functions that calculate basic statistics by iterating over vectors of objects • Use test data to evaluate the correctness of an algorithm • Use exception handling to write robust user input processing code
Topic 5: Object interactions: the Wallet class	Key concepts: You will learn about object interactions, modelling real world items with classes and static and non-static functions. Learning outcomes: • Use class composition to combine classes together which offer higher level functionality • Explain how to model a familiar real-world entity as a class with data and functions • Decide when it is appropriate to use static or non-static functions

Topic 6 Introducing the DJ application and the JUCE toolkit, part 1	Key concepts: You will be learning about GUI libraries, frameworks and tools. Learning outcomes: • Describe the functionality of a DJ application • Explain what a GUI library is and give examples of common GUI widgets • Use the JUCE framework to create and compile a starter application
Topic 7: Event-driven programming and inheritance: buttons, sliders and audio, part 1	Key concepts: You will be learning about JUCE application architecture and event-driven programming. Learning outcomes: • Describe the functions and runtime of a simple JUCE application • Use the JUCE API classes to implement audio playback functionality • Apply event driven programming techniques to create a GUI
Topic 8: Refactoring and class design: the DeckGUI and DJAudioPlayer classes	Key concepts: You will be learning about refactoring into simpler modules and class composition. Learning outcomes: • Identify and refactor audio playback functionality and GUI into a new class • Identify and refactor GUI functionality • Use drag and drop events to add more intuitive file loading functionality

Topic 9: Initialiser lists, constructors and threads: the WaveformDisplay class	Key concepts: You will learn about polymorphism and interfaces. Learning outcomes: • Implement a graphical widget using polymorphism to control module interaction • Make decisions about duplicate data and sharing data inside a program • Use multithreading to implement GUI components that automatically update
Topic 10: Advanced class and user interfaces: using TableListBox and TableListBoxModel	Key concepts: You will learn about linked lists and storing application state. Learning outcomes: • Use advanced JUCE GUI components to implement a table • Reason about data sharing between different parts of a program • Implement a data model for a table component

Learning Activities of This Course

The course is comprised of the following elements:

- **Lecture videos.** In each topic, you will find a sequence of videos in which the example programs for the course are coded up. Further videos review the key programming techniques seen in the coding videos.
- **Readings.** Each topic may include several suggested readings. These are a core part of your learning, and, together with the videos, will cover all of the concepts you need for this course.
- **Practice Quizzes.** Each module will include practice quizzes, intended for you to assess your understanding of the topics. You will be allowed unlimited attempts at each practice quiz. There is no time limit on how long you take to complete each attempt at the quiz. These quizzes do not contribute toward your final score in the class.

- **Programming Activities.** Each topic includes programming activity worksheets. These take you through the steps you have seen in the videos, and provide code excerpts. They also contain challenges activities which challenge you develop the program beyond the functionality seen in the lecture videos.
- **Code.** Each topic includes the C++ code written in the videos. You can use this in combination with the worksheets to ensure you have the correct code.
- **Discussion Prompts.** Each topic includes discussion prompts. You will see the discussion prompt alongside other items in the lesson. Each prompt provides a space for you to respond. After responding, you can see and comment on your peers' responses. All prompts and responses are also accessible from the general discussion forum and the module discussion forum.
- **Assessed coursework.** There are two assessed courseworks, in the middle and at the end of the module. They are based on the two large programs developed during the course.

How to Pass This Course

The module will contain a range of summative and formative assessments. Summative assessments are assessments which contribute directly towards your final grade. Formative assessments do not count directly towards your final grade. Instead, they provide you with opportunities for low stakes practice and will often provide some sort of feedback about your progress. For example, a practice quiz might provide you with feedback about why a particular answer was wrong.

The course has two major assessments each worth 50% of your grade:

- Coursework 1: this consists of a project which will be assessed halfway through course (approximate deadline in week 13)
- Coursework 2: this also consists of a project which will be assessed at the end of the course (approximate deadline in week 22).

There are also several activities that are graded but have 0 weight. That means that they will not count towards your final grade, but they are a key part of your learning, and you need to do them.

This is a detailed breakdown of all the marks:

Activity	Required? (Summative)	Deadline week	Estimated time per course	% of final grade
Written, staff graded coursework 1	Yes	Approximately week 13	25 hours	50%
Written, staff graded coursework 2	Yes	Approximately week 22	25 hours	50%